

AGRICREDX

Blockchain-Backed Attested Receivables Financing for Agricultural Supply Chains

Product Requirements Document (PRD) + Software Requirements Specification
(SRS)

Buildathon positioning

Turn verified agricultural trade evidence into a trusted, financeable on-chain receivable. Blockchain is the shared trust and lifecycle state layer; sensitive documents remain off-chain.

MST Buildathon 2026

BMS College of Engineering

InnoVision

Version 1.2 - Supabase-First Buildathon Specification + Official MST Track Integration (Organizer Guide
Update)

Document Control

Field	Value
Project	AgriCredX
Document type	Combined PRD + SRS
Target environment	MST Testnet; web application
Primary users	Supplier / Exporter, Buyer, Financier, Admin
Primary chain role	Shared trust, attestation, financing state, assignment, repayment, auditability
Sensitive data strategy	Documents and private business data off-chain; hashes / lifecycle state on-chain
Status	Buildathon MVP specification
Version	1.0
Normative language	MUST = mandatory; SHOULD = strongly recommended; MAY = optional

Source Fidelity and Anti-Hallucination Rules

IMPORTANT FOR ANTIGRAVITY

This document is the implementation source of truth for the hackathon MVP. Do not invent chain IDs, RPC URLs, contract addresses, API endpoints, token symbols beyond what is verified, legal claims, financial regulations, or unsupported AI capabilities. Unknown values must remain explicit placeholders until confirmed from official MST documentation or the organizers.

1. Preserve the canonical project concept: AgriCredX creates an Attested Receivable from Invoice + PO + GRN + quality evidence + buyer acceptance and makes its trust state financeable on MST.
2. Treat AI as an analysis and extraction assistant, not the final authority for buyer acceptance or financing approval.
3. Do not store full invoices, PDFs, lab reports, or private business information on-chain. Store only hashes, identifiers, state, timestamps, wallets, and contract events required for shared trust.
4. Do not claim the prototype is a regulated financial product. The build is a testnet hackathon prototype; no real-money lending is required or authorized by this specification.

5. Do not fabricate integrations. If MST network parameters or BridgeKey behavior are not available in the build environment, use clearly marked environment variables and an integration stub rather than making up values.
6. Do not silently change the four-part lifecycle model. A hackathon simplification is allowed only when explicitly labeled as MVP simplification and must preserve the same business semantics.
7. Every external document uploaded by a user must have a deterministic content hash. Re-uploading the exact same bytes should reproduce the same hash.
8. Every blockchain write MUST be attributable to a wallet address and mapped to an application audit record.
9. Supabase is the application backend of record for private business data. RLS MUST be enabled on every user-facing table before application data is exposed.
10. Supabase service-role credentials may be used only in server-side/Edge Function secrets and MUST never enter React/Vite browser code.
11. Python FastAPI is AI-only. Do not silently turn it into a second general CRUD backend unless this specification is explicitly updated.
12. Use official library APIs only. When uncertain about Supabase, ethers.js, Hardhat, OpenZeppelin, PyMuPDF, Tesseract or Gemini behavior, verify documentation before implementation instead of guessing.
13. Never claim a feature is integrated or working until it has been observed end-to-end with evidence such as a real Supabase record or confirmed MST testnet transaction.
14. Demo organizations, identities and financing records are fictional. Label seed data as DEMO DATA and never present it as a real financial transaction.
15. The official MST Blockchain Buildathon Track Guide is authoritative for prize eligibility, mandatory submission artifacts, official developer resources and organizer-provided tooling. This PRD/SRS remains the source of truth for AgriCredX product architecture, data model and implementation behavior.

Canonical External Facts to Verify at Build Time

Fact	Current official source status	Implementation rule
MST Blockchain / EVM-compatible target chain	Organizer track guide requires meaningful MST usage and MST Testnet deployment; official docs/resources are provided.	Use verified MST/EVM patterns; do not invent chain parameters.
MST testnet faucet exists	Organizer guide points to https://faucet.masterstroke.academy .	Acquire test MSTC and verify the faucet at build time.
MSTScan explorer exists	Organizer guide points to https://mstscan.com .	Surface real explorer links only after confirmed testnet transactions.
BridgeKey is official MST wallet	Organizer guide strongly recommends BridgeKey; current BridgeKey site describes a non-custodial MST-native EVM wallet.	Target BridgeKey as primary wallet; approved EVM fallback only if needed.

Fact	Current official source status	Implementation rule
Performance figures	MST publishes its own performance figures on its current site.	Do not make the product depend on a specific TPS or block-time value; performance claims must be presented as vendor-published if mentioned.
Official MST SDKs	Organizer provides @mstblockchain/mst-sdk (TypeScript) and mst-sdk-python (Python).	Prefer the TypeScript SDK for MST-specific web/client interaction; keep Python SDK optional and outside FastAPI AI runtime.
VibeKit SDK	Organizer provides @mstblockchain/mst-vibe-kit for rapid MST scaffolding/deployment.	Use as bootstrap/dev tooling if it accelerates setup without changing architecture.
MCP endpoint	Organizer provides https://mcp.mstblockchain.com/sse for developer documentation access.	Optional developer tool only; never runtime or source-committed credentials.

0. MST Buildathon 2026 Track Compliance & Official Developer Resources

The organizer-provided MST Blockchain Buildathon Track Guide introduces mandatory prize-eligibility requirements and an official set of developer resources. These requirements do not replace the AgriCredX architecture; they strengthen the deployment, proof, submission and MST-integration portions of this specification.

0.1 Mandatory / High-Priority Track Requirements

- Primary track alignment: Category 04 - Real-World Assets & Supply Chain, especially agriculture traceability, supply-chain tracking and logistics/delivery proof. Secondary relevance: programmable commerce, identity/trust, and finance.

Compliance posture: the track guide states the five listed requirements are part of prize eligibility, while items 4 and 5 are themselves labelled strongly recommended/preferred. InnoVision will treat BridgeKey and a fully interactive on-chain demo as build priorities anyway, so the safest implementation target is the strongest interpretation of the guide.

- Meaningful MST usage is mandatory for prize eligibility. A wallet connection or one isolated transaction is insufficient; MST must be integral to the product. AgriCredX therefore uses MST for buyer acceptance, attestation anchoring, financing state/assignment, funding state, repayment state and auditable lifecycle events.
- MST Testnet deployment is mandatory before submission. The final submission must include at least one deployed MST Testnet contract address, at least one verifiable MST Testnet transaction hash, and a working application/demo link.
- A public GitHub repository is mandatory. It must contain the smart contracts plus frontend/backend source code, and a README with MST integration details, contract addresses and setup instructions.

- BridgeKey is the official non-custodial MST wallet and is strongly recommended by the organizer. AgriCredX will target BridgeKey as the primary wallet. An approved EVM wallet is a documented fallback only if BridgeKey integration is blocked in the build environment.
- A working deployed product with verifiable on-chain proof is strongly preferred. The judge flow must expose real MST Testnet transactions rather than fabricated transaction data.
- AI coding assistants, AI agents, MCP, VibeKit, official SDKs and open-source libraries are permitted. The team must understand and be able to explain submitted code and MST integration. Fake or misleading deployment/transaction data is prohibited and may cause disqualification.

0.2 Official MST Developer Resources

Resource	Official location	Use in AgriCredX	Decision
Official Docs	https://docs.mstblockchain.com	Authoritative MST/EVM/network/deployment reference.	MUST use for verification
BridgeKey Wallet	https://bridgekey.io	Primary wallet target for buyer/supplier/financier signing.	STRONGLY RECOMMENDED
BridgeKey Extension	https://chromewebstore.google.com/detail/bridgekey/bfjojdcfenehemjgilepdjomkpginlkg	Browser wallet connection and transaction signing.	Test at Hour 0
MST Testnet Faucet	https://faucet.masterstroke.academy	Acquire test MSTC for deployment and demo transactions.	MUST use/test
TypeScript SDK	https://www.npmjs.com/package/@mstblockchain/mst-sdk	Primary official JS/TS MST client where it supports required contract/provider flows.	RECOMMENDED / preferred chain client
Python SDK	https://pypi.org/project/mst-sdk-python/	Available for standalone chain scripts/utilities when useful.	OPTIONAL; not used by AI service for blockchain writes
VibeKit SDK	https://www.npmjs.com/package/@mstblockchain/mst-vibe-kit	Rapid MST project scaffolding and deployment acceleration.	RECOMMENDED for initial setup if compatible
MCP Endpoint	https://mcp.mstblockchain.com/sse	Developer-time access to MST documentation and knowledge.	OPTIONAL; developer tool, not runtime
MSTScan Explorer	https://mstscan.com	Public verification of live MST transactions and contract activity.	MUST surface live proof

0.3 Integration Decisions for InnoVision

- Keep Solidity + Hardhat + OpenZeppelin for contract development and tests. Use the official MST TypeScript SDK as the preferred MST-specific client integration layer where its documented APIs cover the required provider, contract and transaction flows.
- Tool selection rule: use the official MST SDK/VibeKit only where they reduce integration risk and their documented APIs cover the required behavior; otherwise keep the existing, tested EVM tooling. Never rewrite the architecture merely to use an SDK.
- Retain ethers.js only where needed for standard EVM/ABI/wallet-provider compatibility. Do not let generic EVM assumptions override verified MST SDK/documentation behavior.

- Use VibeKit as a bootstrap/deployment accelerator only; it must not force a change to the Supabase-first architecture or become a mandatory runtime dependency.
- The Python SDK is not part of the FastAPI AI service. FastAPI remains AI-only; any Python MST SDK use belongs in a separate deployment/verification utility if genuinely needed.
- MCP is a developer productivity tool for reading official MST documentation. It is not an application runtime dependency and no MCP credential belongs in source control.
- MSTScan is the external proof layer: every important on-chain action must resolve to a real transaction hash and an explorer link after successful testnet confirmation.

0.4 Submission Artifact Checklist

- Public GitHub repository URL
- Deployed MST Testnet contract address(es)
- At least one verifiable MST Testnet transaction hash; preferably multiple hashes covering acceptance, attestation, funding and/or repayment
- Working demo/application URL where applicable
- Demo video / backup recording
- README containing MST integration details, contract addresses, environment/setup instructions and verified testnet proof

Table of Contents

- 0. MST Buildathon 2026 Track Compliance & Official Developer Resources
- 1. Executive Summary
- 2. Problem Statement and Opportunity
- 3. Product Vision, Goals and Success Criteria
- 4. Users, Roles and Trust Model
- 5. Canonical Case Scenario
- 6. Product Scope: MVP, Stretch and Non-Goals
- 7. Key Concepts and Terminology
- 8. End-to-End User Journeys
- 9. Functional Requirements
- 10. Non-Functional Requirements
- 11. System Architecture
- 11.2 Implementation Stack - Supabase First
- 12. On-Chain / Off-Chain Boundary
- 13. Data Model and Storage Requirements
- 14. Smart Contract Specification
- 15. API and Supabase Backend Specification
- 16. AI and Risk Engine Specification
- 17. Document Integrity and Tamper Detection
- 18. Attestation and Buyer Acceptance
- 19. Financing Marketplace
- 20. Repayment and Closure
- 21. Frontend / UX Specification
- 22. Security Threat Model
- 23. Privacy and Compliance Guardrails
- 24. Auditability and Observability

- 25. Test Strategy and Acceptance Criteria
- 26. Judge Demo Script
- 26.1 Unique Selling Points (USPs)
- 27. 24-Hour Delivery Plan
- 28. Team Responsibilities
- 29. Deployment and Runbook
- 30. Roadmap and Future Extensions
- 31. Reference Architecture and External Sources
- 32. Final Antigravity Build Checklist

1. Executive Summary

1.1 What AgriCredX Is

AgriCredX is a blockchain-backed receivables financing platform for agricultural supply chains. A supplier may hold a legitimate unpaid invoice while waiting 30-90 days for payment. The system converts supporting trade evidence into a machine-verifiable and buyer-attested receivable, anchors the trust state on MST, and makes that receivable available to a financing marketplace.

The concept is not “put invoices on blockchain.” The core idea is to put the shared proof and financial lifecycle state on blockchain while keeping sensitive business documents off-chain. This distinction is the central architectural principle.

1.2 Core Value Proposition

ONE-LINE VALUE PROPOSITION

Verify the trade. Attest the receivable. Finance it on-chain.

Problem	AgriCredX response
Fragmented proof across PDF, email, ERP and logistics records	AI extraction + deterministic cross-checks create one verification report.
Financier cannot easily trust that an invoice is genuine	Buyer acceptance + document hashes + on-chain attestation provide shared evidence.
Risk of invoice duplication or document manipulation	Duplicate checks and content hashes detect repeat or altered submissions.
Supplier waits for payment while holding a valid receivable	Verified receivable enters a financing marketplace.
No common lifecycle state across supplier, buyer, financier	MST records attestation, funding and repayment state in an independently auditable layer.

2. Problem Statement and Opportunity

2.1 Problem Statement

Agricultural suppliers and exporters may have valid trade receivables but limited working capital. A receivable can be economically valuable while remaining difficult to finance because the evidence supporting it is fragmented and the parties do not share a common trust layer.

- Invoice authenticity is not always directly connected to the purchase order that created the obligation.

- Goods receipt and delivery evidence may be stored in separate systems.
- Quality certificates and lab reports are often separate attachments.
- Buyer acceptance may live in email or a private ERP workflow.
- A document can be altered after issuance while retaining the same filename and business identifier.
- A receivable could be presented to more than one financing party unless there is a shared state and assignment record.

2.2 Why Blockchain Is Necessary

A conventional database remains appropriate for private documents and operational data. The reason to use MST is the multi-party trust boundary: supplier, buyer, financier and platform each have different incentives. The blockchain should hold only the state that multiple parties need to verify independently, such as document digests, wallet identities, attestation status, financing status, assignment, repayment and lifecycle timestamps.

2.3 Why AI Is Necessary

The source material defines AI as an analysis assistant. The AI layer extracts fields from documents, compares fields across documents, flags inconsistencies, detects likely duplicates or tampering, and produces an explainable verification report. The business participant remains responsible for the final attestation.

3. Product Vision, Goals and Success Criteria

3.1 Product Vision

Build a verifiable bridge between agricultural trade evidence and working-capital finance: a receivable should become financeable only after its underlying trade can be traced, checked, accepted and anchored to an auditable lifecycle state.

3.2 Product Goals

1. Create a canonical receivable record from invoice and supporting trade documents.
2. Extract and cross-check data automatically while keeping the logic explainable.
3. Generate a deterministic attestation digest for the verified receivable.
4. Anchor the attested state and lifecycle transitions on MST Testnet.
5. Let the buyer explicitly accept or dispute the receivable.
6. Let financiers submit financing offers against only financeable receivables.
7. Record funding, outstanding and repayment state on-chain.
8. Provide a visible tamper-detection demo by modifying a document and showing the integrity mismatch.
9. Produce an end-to-end demo that is reliable within a 24-hour buildathon environment.

3.3 Buildathon Success Criteria

Metric	Target for MVP
End-to-end workflow	One complete happy path from upload to repayment.
Blockchain integration	At least one live MST Testnet contract deployment and multiple state-changing transactions visible on MSTScan.
AI verification	At least invoice/PO/GRN extraction and 3-5 deterministic cross-field checks.
Tamper demo	Changing a canonical field must produce a different document digest and block financeability.
Traceability	Any financeable receivable can display its source documents, verification report, wallet identities and on-chain events.
Security	No private keys embedded in source code; no sensitive documents committed to Git; role checks enforced in contract/backend.

4. Users, Roles and Trust Model

Role	Primary actions	What they trust	What they must sign
Supplier / Exporter	Create receivable, upload evidence, request financing, choose bid, view repayment	Verification report and financing state	Receivable submission / financing choice as applicable
Buyer	Review trade evidence, accept or dispute, repay	Invoice/PO/GRN consistency and delivered quantity	Buyer acceptance and repayment
Financier	Browse financeable receivables, submit offer, fund selected receivable	Attested state, buyer acceptance, non-duplication	Offer / funding transaction
Admin	System operations, dispute visibility, audit review; no silent mutation of chain state	Audit logs and contract events	Administrative actions only where explicitly permitted
AI Engine	Extract and compare data; produce risk signals	Input document bytes and configured rules	None; AI does not attest or release funds

TRUST RULE

The platform may orchestrate workflows, but the final authority for trade acceptance is a participant. The blockchain is the shared state layer, not an oracle that independently knows whether goods physically exist.

5. Canonical Case Scenario

The following scenario is the canonical demo case and should be treated as the reference dataset throughout the build unless the team explicitly replaces it.

Field	Canonical value
Invoice ID	INV-2026-09124
Commodity	Basmati Rice
Invoice amount	INR 8,50,000
Payment term	60 days
Requested financing	INR 8,20,000
Buyer	ABC Foods (demo entity)
Supplier	Demo Basmati Exporter
Financiers	Financier A, Financier B, Financier C
Initial AI risk example	18/100 - LOW



Figure 1 - Canonical AgriCredX lifecycle from trade evidence to repayment.

5.1 Happy Path

1. Supplier logs in and creates INV-2026-09124 for INR 8,50,000 with a 60-day due date.
2. Supplier uploads invoice, purchase order, goods receipt note, and quality evidence.
3. AI extracts structured fields and deterministic rules compare amount, buyer, quantity, identifiers and dates.

4. System produces a verification report and marks the receivable ready for buyer review.
5. Buyer reviews the summary and signs acceptance through the wallet.
6. System creates the attestation digest from the canonical record and document hashes and anchors the digest on MST.
7. Receivable becomes financeable. Financiers submit financing offers.
8. Supplier selects a financing offer. The testnet funding transaction is executed.
9. Later, the buyer submits repayment. The repayment manager records REPAID and the receivable transitions to CLOSED.

5.2 Tamper Path

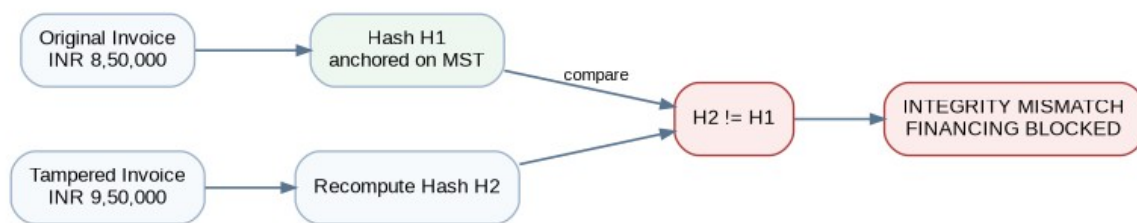


Figure 2 - Canonical tamper demonstration: changing INR 8,50,000 to INR 9,50,000 must break integrity.

6. Product Scope: MVP, Stretch and Non-Goals

6.1 MUST Ship in MVP

- Role-based web UI for supplier, buyer and financier.
- Invoice/PO/GRN upload.
- Document hash generation and persistence.
- AI/OCR field extraction with explainable verification output.
- Deterministic consistency checks.
- Buyer acceptance flow.
- On-chain attestation record.
- Financing request and one working bid flow.
- On-chain funding status.
- Repayment status.
- MSTScan transaction link.
- Tamper-detection test.

6.2 SHOULD Ship if Time Permits

- Multiple competing financing bids.
- QR-based receivable verification page.
- Traceability graph from farm -> batch -> quality certificate -> exporter -> invoice -> buyer -> financier -> repayment.
- Dispute state with admin visibility.

- Document version history.
- Optional IPFS/object-storage CID for larger evidence packages; not required for MVP if object storage is sufficient.

6.3 OUT OF SCOPE

- Real fiat lending or real-money financing.
- Production credit underwriting or credit scores that claim regulatory validity.
- Real-world automated disbursement to bank accounts.
- Legal enforceability of on-chain receivable ownership.
- Real farm/ERP integrations unless a mock adapter is available.
- Full KYC/AML product implementation.
- Mainnet launch.

7. Key Concepts and Terminology

Term	Definition
Receivable	A buyer obligation to pay the supplier for an accepted trade transaction.
Attested Receivable	A receivable for which supporting evidence has been checked, the relevant business participant has accepted it, and a cryptographic digest has been anchored to MST.
Document Hash	A deterministic cryptographic digest of the exact uploaded bytes.
Attestation Digest	A canonical digest over the receivable identity, participant wallets, selected document hashes, verification flags and timestamp/version.
Financeable	A lifecycle state meaning the receivable has passed required verification and buyer acceptance and can appear in the financing marketplace.
Assignment	The on-chain record of which financier currently funds the receivable. An active receivable MUST NOT be simultaneously marked funded by multiple financiers.
Lifecycle State	The canonical state machine used by the application and smart contracts.
Evidence Bundle	The set of invoice, PO, GRN and quality/delivery documents supporting the receivable.

8. End-to-End User Journeys

Supplier journey

1. Create organization profile -> create invoice -> upload evidence -> run verification -> review flags -> submit to buyer -> monitor acceptance -> open financing -> compare offers -> select offer -> track funding -> track repayment.

Buyer journey

1. Open pending receivable -> inspect invoice summary and source documents -> review verification checks -> accept or dispute -> sign acceptance -> later review repayment notice -> sign repayment.

Financier journey

1. Open financeable receivables -> inspect attestation, buyer acceptance and risk signals -> submit offer -> fund selected receivable -> track outstanding state -> confirm repayment.

Admin journey

1. Monitor integration status -> inspect audit log -> resolve platform configuration issues -> review disputes -> never rewrite chain state manually.

9. Functional Requirements

ID	Area	Requirement
FR-001	Authentication	System MUST support authenticated sessions for Supplier, Buyer, Financier and Admin roles.
FR-002	RBAC	System MUST prevent a user from performing actions not permitted for their role.
FR-003	Create receivable	Supplier MUST be able to create a receivable with invoice ID, amount, due date, currency, buyer and commodity.
FR-004	Document upload	Supplier MUST upload Invoice, PO and GRN; Quality Certificate/Lab Report SHOULD be supported.
FR-005	Content hashing	Every uploaded document MUST receive a SHA-256 or equivalent content digest before storage.

ID	Area	Requirement
FR-006	Metadata extraction	System MUST extract invoice number, supplier, buyer, amount, quantity and date fields where present.
FR-007	Cross-check	System MUST compare invoice amount vs PO amount, invoice buyer vs PO buyer, invoice quantity vs GRN quantity, and other configured checks.
FR-008	Duplicate detection	System MUST detect repeated invoice IDs and/or repeated exact document hashes.
FR-009	Verification report	System MUST present each verification check as PASS / WARN / FAIL with an explanation.
FR-010	AI limitation	AI MUST NOT directly move a receivable to buyer-accepted or funded state.
FR-011	Buyer review	Buyer MUST view evidence summary and verification report.
FR-012	Buyer acceptance	Buyer MUST be able to sign acceptance; the accepted state MUST be linked to a wallet address and transaction hash.
FR-013	Buyer dispute	Buyer SHOULD be able to dispute with a reason; disputed state MUST block new financing offers.
FR-014	Attestation creation	System MUST create an attestation payload from canonical fields and hashes.
FR-015	On-chain anchoring	System MUST anchor the attestation digest and lifecycle state on MST Testnet.
FR-016	Financeability rule	Only ATTESTED + BUYER_ACCEPTED receivables MAY become FINANCEABLE.
FR-017	Marketplace	Supplier SHOULD be able to open a financing request with requested amount and desired maturity.

ID	Area	Requirement
FR-018	Financier bid	Financier MUST be able to submit a testnet financing offer.
FR-019	Selection	Supplier MUST be able to select exactly one active financing offer.
FR-020	Assignment lock	Once FUNDED, system MUST block another financier from funding the same receivable.
FR-021	Funding record	Funding transaction hash MUST be stored off-chain and emitted on-chain as an event or state transition.
FR-022	Repayment	Buyer MUST be able to create a repayment transaction in the testnet environment.
FR-023	Closure	System MUST mark the receivable REPAID and then CLOSED after successful repayment confirmation.
FR-024	Audit log	Every privileged action and every chain transaction MUST be recorded in an application audit log.
FR-025	Explorer link	UI MUST provide an MSTScan link for each on-chain transaction.
FR-026	Tamper detection	If any bound document changes, its new hash MUST no longer match the attested digest.
FR-027	Tamper blocking	Integrity mismatch MUST block the receivable from becoming financeable.
FR-028	Versioning	System SHOULD retain previous document versions and identify the active version.
FR-029	Traceability	System SHOULD show the chain of trade entities and states for a receivable.

ID	Area	Requirement
FR-030	Export	System SHOULD export a verification report as JSON/PDF for demo/audit purposes.

10. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Reliability	Happy-path demo MUST complete end-to-end without manual database edits.
NFR-002	Determinism	Hashing and deterministic business rules MUST produce stable results for identical inputs.
NFR-003	Security	No private keys or credentials in source control, frontend bundle, logs or screenshots.
NFR-004	Traceability	Every state change MUST be attributable to a user action or contract transaction.
NFR-005	Performance	Document extraction for demo documents SHOULD complete within a practical interactive window; no strict SLA is required for the hackathon.
NFR-006	Usability	A first-time judge SHOULD be able to follow the workflow without developer assistance.
NFR-007	Blockchain isolation	The application MUST function with explicit network configuration supplied through environment variables.
NFR-008	Data separation	Sensitive documents MUST remain off-chain.
NFR-009	Auditability	Contract events and transaction hashes MUST be queryable from the application.

ID	Category	Requirement
NFR-010	Maintainability	Contract interfaces, API models and database schema MUST be versioned and documented.
NFR-011	Failure handling	RPC failures, rejected signatures and malformed documents MUST show actionable user-facing errors.
NFR-012	Reproducibility	The canonical demo dataset MUST be seedable from a script so the entire demo can be reset.

11. System Architecture

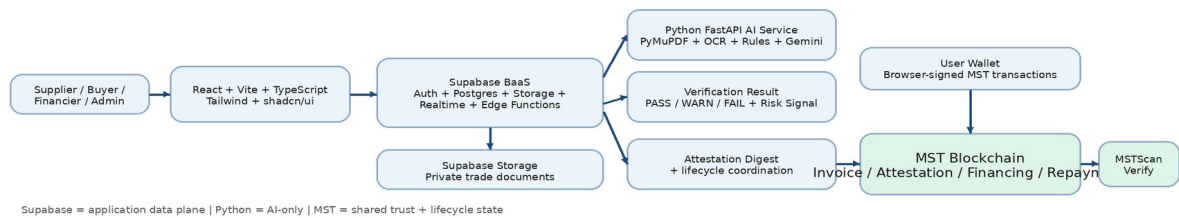


Figure 3 - Reference system architecture. Sensitive data stays off-chain; shared trust state is anchored on MST.

11.1 Component Responsibilities

Component	Responsibility	MUST NOT do
React + Vite UI	Role-based dashboards, uploads, verification presentation, wallet interactions, financing marketplace, explorer links.	Never hold private keys, backend secrets, or privileged service credentials.
Supabase BaaS	Primary application backend: Postgres, Auth, Storage, Realtime, Edge Functions, RLS, audit data and workflow coordination.	Never be treated as the final shared trust authority for state committed on MST.
Supabase Storage	Private storage for Invoice PDFs, POs, GRNs, quality certificates, lab reports and evidence packages.	Never expose private documents publicly without authorization.
Python FastAPI AI microservice	Document parsing, OCR, typed extraction, deterministic cross-checks, anomaly/tamper signals, explainable risk report.	Never authorize funding, buyer acceptance, or execute blockchain commands.

Supabase Edge Functions	Privileged coordination, AI-service invocation, audit-log writes, controlled server-side operations and integration glue.	Never hold or expose end-user wallet private keys; never fabricate chain state.
MST Smart Contracts	Shared lifecycle state, attestation digest anchoring, financeability, assignment, funding and repayment rules.	Never store full documents or depend on LLM output.
Wallet	Signs user-approved MST transactions from the browser/client.	Never send seed phrases or private keys to Supabase or the AI service.
MSTScan	Public explorer used to independently verify live testnet transactions.	Never be treated as an application database or source of private business data.

11.2 Implementation Stack - Supabase First

Supabase is the PRIMARY application backend for the MVP. React communicates with Supabase for authentication, Postgres data, Storage and Realtime. Supabase Edge Functions provide privileged coordination. Python/FastAPI is a separate AI-only microservice because PDF/OCR/CV libraries require a Python runtime. User blockchain transactions remain wallet-controlled in the client; MST is the shared trust and lifecycle layer.

Layer	Technology	Why / Rule
Frontend	React 18 + Vite + TypeScript	Fast SPA build; familiar team stack; no separate Next.js server layer.
UI	Tailwind CSS + shadcn/ui	Consistent professional interface under hackathon time pressure.
Client data	supabase-js + TanStack Query	Typed data access, caching and realtime-friendly queries.
Primary backend	Supabase: Postgres + Auth + Storage + Realtime + Edge Functions	Single low-friction application backend with RLS and serverless coordination.
AI service	Python + FastAPI + PyMuPDF + OpenCV + Tesseract + Gemini API	Document parsing/OCR, typed extraction, deterministic checks and explainable risk reporting.
Blockchain	MST Testnet + Solidity + Hardhat + @mstblockchain/mst-sdk + OpenZeppelin; ethers.js only where standard EVM/wallet compatibility is required	Official MST client integration plus standard EVM contract tooling; MST Testnet is the shared trust/lifecycle layer.
Wallet	BridgeKey primary; approved EVM wallet fallback	BridgeKey is strongly recommended by the organizer; user-controlled signing, private keys remain client-side.

Hosting	Vercel/Netlify + Supabase Cloud + Render/Fly.io	Zero-ops deployment suitable for a 24-hour buildathon.
MST developer tooling	@mstblockchain/mst-vibe-kit	Rapid scaffold/deployment accelerator; optional after bootstrap and not a runtime dependency.
MST documentation tooling	Official Docs + MCP endpoint	Docs are authoritative; MCP is optional developer-time documentation access only.

12. On-Chain / Off-Chain Boundary

Off-chain	On-chain
Invoice PDF, PO, GRN, certificates, lab reports, images	Invoice ID
User profile, organization data, private addresses	Supplier wallet / Buyer wallet / Financier wallet
AI extraction raw text and detailed reasoning	Document hashes / attestation digest
Risk report details	Lifecycle state
Private financing terms beyond what is needed for shared state	Financing status / selected financier / funded amount as agreed for MVP
Audit log details and application telemetry	State-change events / timestamps / tx hashes

HASHING RULE

Hash the exact uploaded bytes. If a PDF changes by one byte, the raw content hash must change. If business normalization is needed, keep a second canonical JSON digest separate from the raw file hash.

13. Data Model and Storage Requirements

Entity	Required fields
User	id, role, email, organization_id, wallet_address, created_at
Organization	id, name, type, contact metadata
Receivable	id, invoice_id, supplier_id, buyer_id, amount, currency, due_date, commodity, status, attestation_digest, created_at

Entity	Required fields
Document	id, receivable_id, type, filename, mime_type, sha256, object_key, version, uploaded_by, uploaded_at
VerificationResult	id, receivable_id, check_code, status, extracted_value, expected_value, explanation, engine_version, created_at
Attestation	id, receivable_id, digest, signer_wallet, tx_hash, block_reference, timestamp, schema_version
FinancingRequest	id, receivable_id, requested_amount, desired_term, status, created_at
FinancingBid	id, request_id, financier_id, amount, fee_or_discount, maturity, status, tx_hash
Financing	id, receivable_id, financier_id, funded_amount, funded_at, status
Repayment	id, receivable_id, amount, paid_at, tx_hash, status
AuditLog	id, actor_id, action, entity_type, entity_id, metadata_json, created_at

13.1 Canonical Attestation Payload

[json]

```
{
  "schemaVersion": "1.0",
  "invoiceId": "INV-2026-09124",
  "supplierWallet": "0x...",
  "buyerWallet": "0x...",
  "invoiceAmount": 850000,
  "currency": "INR",
  "dueDate": "<ISO-8601>",
  "poHash": "0x...",
  "invoiceHash": "0x...",
  "grnHash": "0x...",
  "qualityEvidenceHash": "0x...",
  "poMatched": true,
  "grnMatched": true,
  "buyerAccepted": true,
  "duplicateCheckPassed": true,
  "verificationEngineVersion": "v1",
  "createdAt": "<ISO-8601>"
}
```

The exact serialization order and encoding **MUST** be fixed in code and tests before contract deployment so that the same logical attestation always produces the same digest.

14. Smart Contract Specification

The source concept defines four primary contracts. For the hackathon, the team may merge them physically into fewer contracts while preserving the same logical boundaries. The logical responsibilities below are normative.

Contract	Responsibilities	Critical invariant
InvoiceRegistry	Create receivable; store core identifiers, amount, due date, participant addresses, current state; emit lifecycle events.	Only supplier or authorized server relayer can create; state transitions checked.
AttestationRegistry	Store attestation digest and verification status; bind digest to receivable ID; provide read functions.	No acceptance based on LLM text; signer role must be verified.
FinancingPool	Open financing request, receive one selected fund, block duplicate funding, expose financier and funded amount.	A receivable cannot be funded unless financeability conditions hold.
RepaymentManager	Record repayment, transition to REPAID and CLOSED, expose repayment history.	Repayment can only apply to FUNDED/OUTSTANDING receivable.

14.1 Canonical State Machine

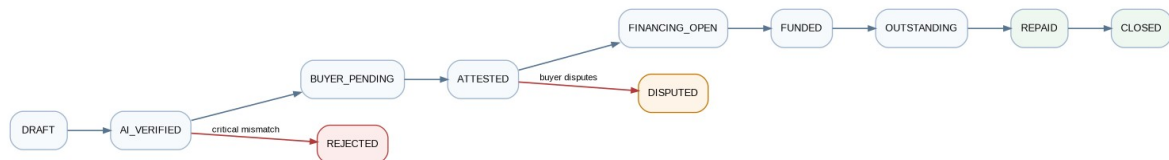


Figure 4 - Canonical receivable state machine. Reject/dispute paths block financeability until resolved.

14.2 Suggested Function Surface

[solidity interface]

// Illustrative interface only - exact Solidity implementation may vary.

```

createReceivable(
  bytes32 invoiceId,
  address buyer,
  uint256 amount,
  uint64 dueDate
) returns (uint256 receivableId)

```

```

anchorAttestation(
  uint256 receivableId,
  bytes32 attestationDigest
)

```

)

```

acceptReceivable(uint256 receivableId)
disputeReceivable(uint256 receivableId, bytes32 reasonHash)

openFinancing(uint256 receivableId, uint256 requestedAmount)
submitBid(uint256 financingRequestId, uint256 amount, uint256 termsHash)
selectBid(uint256 bidId)
fund(uint256 financingId)

repay(uint256 receivableId, uint256 amount)
closeReceivable(uint256 receivableId)

getReceivable(uint256 receivableId)
getLifecycle(uint256 receivableId)
getAttestation(uint256 receivableId)

```

14.3 Smart Contract Invariants

- A receivable ID MUST be unique.
- An invoice marked REJECTED or DISPUTED MUST NOT be financeable.
- A receivable MUST NOT be funded twice.
- Only the configured buyer wallet (or explicitly authorized buyer role) can accept the receivable.
- Only the selected financier can hold the funded assignment.
- Repayment MUST NOT exceed the recorded outstanding obligation unless overpayment handling is explicitly implemented.
- State transitions MUST be monotonic except for an explicitly defined dispute/review transition.
- Contract events MUST emit enough information for the frontend to reconstruct the lifecycle.

15. API and Supabase Backend Specification

The logical API contract below is the application workflow contract. In the Supabase-first MVP, CRUD can use supabase-js/PostgREST under RLS, while privileged or multi-step operations are implemented as Supabase Edge Functions. The Python FastAPI service is AI-only and is not the primary CRUD backend.

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate demo user.
GET	/api/receivables	List receivables with role-aware filters.
POST	/api/receivables	Create draft receivable.
POST	/api/receivables/{id}/documents	Upload a document.
POST	/api/receivables/{id}/verify	Run extraction + verification.
GET	/api/receivables/{id}/verification	Get verification report.

Method	Endpoint	Purpose
POST	/api/receivables/{id}/submit-to-buyer	Move to buyer review.
POST	/api/receivables/{id}/accept	Record buyer acceptance and chain tx reference.
POST	/api/receivables/{id}/dispute	Record buyer dispute.
POST	/api/receivables/{id}/attest	Create/anchor attestation.
POST	/api/financing-requests	Open financing request.
POST	/api/financing-requests/{id}/bids	Submit financing bid.
POST	/api/financing-requests/{id}/select	Select bid.
POST	/api/financings/{id}/fund	Initiate funding flow.
POST	/api/receivables/{id}/repay	Initiate repayment.
GET	/api/transactions/{hash}	Get transaction status / explorer link.
GET	/api/audit/{entityType}/{entityId}	Get audit trail.

15.1 API Rules

- All endpoints MUST validate role and ownership.
- All state-changing requests MUST be idempotency-safe where practical.
- Chain writes SHOULD be asynchronous from the UI: show PENDING until transaction receipt is confirmed.
- API errors MUST use stable error codes, e.g. DOCUMENT_TAMPERED, DUPLICATE_INVOICE, BUYER_SIGNATURE_REJECTED, RPC_UNAVAILABLE.
- The frontend MUST NOT fabricate transaction hashes or “confirmed” states.

16. AI and Risk Engine Specification

16.1 Pipeline

1. Load document bytes from object storage.
2. Extract text using a PDF/text parser and OCR only when needed.
3. Normalize fields into a typed JSON schema.
4. Run deterministic field-to-field checks.

5. Run duplicate and anomaly checks.
6. Generate a human-readable verification report with evidence for every flag.
7. Persist engine version and result hash.

Supabase integration rule: the UI requests verification through an Edge Function; the Edge Function fetches private documents from Supabase Storage, calls the Python AI service, validates typed output, persists VerificationResult rows, and publishes status via Supabase Realtime.

16.2 Minimum Deterministic Checks

Check code	Inputs	PASS condition
AMOUNT_MATCH	Invoice amount, PO amount	Equal after currency/decimal normalization.
BUYER_MATCH	Invoice buyer, PO buyer	Normalized legal/entity names match expected rule.
QTY_MATCH	Invoice quantity, GRN quantity	Within configured exact or tolerance rule; for MVP use exact match.
DATE_CONSISTENCY	Invoice date, PO date, GRN date	Sequence satisfies configured business rule.
DUPLICATE_ID	Invoice ID against existing receivables	No active duplicate.
DOCUMENT_HASH	Current hash vs attested hash	Equal for unchanged document version.

16.3 Risk Output

```
[json]
{
  "riskScore": 18,
  "riskBand": "LOW",
  "checks": [
    {"code": "AMOUNT_MATCH", "status": "PASS", "explanation": "Invoice and PO amount match."},
    {"code": "QTY_MATCH", "status": "PASS", "explanation": "Invoice and GRN quantity match."},
    {"code": "DUPLICATE_ID", "status": "PASS", "explanation": "Invoice ID is not already financed."}
  ],
  "engineVersion": "v1"
}
```

ANTI-HALLUCINATION RULE

LLM output MUST be treated as untrusted text until converted into typed fields and checked against deterministic rules. The contract MUST never execute an LLM-generated command or accept an LLM claim as proof by itself.

17. Document Integrity and Tamper Detection

17.1 Raw File Integrity

When a file is uploaded, compute SHA-256 over the exact bytes and store it with the document version. The hash becomes a stable identity for that exact artifact.

17.2 Canonical Attestation Integrity

The attestation digest is computed from a canonical serialization of the receivable record plus the selected document hashes and verification flags. This makes the trust state independently reproducible.

17.3 Tamper Demo

- Start with the canonical invoice at INR 8,50,000 and create the attestation.
- Change only the amount to INR 9,50,000.
- Recompute the document hash and/or canonical digest.
- Show $H2 \neq H1$.
- Block financeability and display INTEGRITY MISMATCH / DOCUMENT TAMPERING DETECTED.

18. Attestation and Buyer Acceptance

18.1 Attestation Preconditions

- Required documents uploaded.
- Minimum required checks PASS.
- No unresolved duplicate invoice.
- No document integrity mismatch.
- Buyer is the configured buyer participant.

18.2 Buyer Acceptance Transaction

The buyer signs a transaction that commits acceptance of the specific receivable ID and its current attestation digest. The backend MUST store the transaction hash and the signed state change.

18.3 Dispute

A dispute MUST stop the financing flow. A disputed receivable remains visible for resolution but cannot transition to FINANCEABLE until an explicit resolution action clears the dispute under the configured rules.

19. Financing Marketplace

19.1 Marketplace Rule

Only receivables that are both attested and buyer-accepted can appear as financeable opportunities.

19.2 Bid Model

Field	Description
bidId	Unique bid ID
financierWallet	Wallet address
offerAmount	Amount financier will fund
feeOrDiscount	Demo financing economics field
maturityDate	Expected repayment date
termsHash	Digest of the financing terms
status	OPEN / SELECTED / REJECTED

19.3 Selection and Funding

- Supplier selects exactly one OPEN bid.
- Selection creates an assignment record.
- Funding moves the receivable to FUNDED.
- Any second attempt to fund MUST fail or be rejected as already funded.

20. Repayment and Closure

1. Buyer opens outstanding receivable.
2. System displays outstanding amount, financier and due date.
3. Buyer signs repayment transaction.
4. RepaymentManager records the payment and emits REPAID event.
5. System updates application state.
6. Receivable is marked CLOSED after the required final checks.

IMPORTANT

This is a testnet prototype. Do not imply that the hackathon contract creates a legally enforceable receivable assignment, regulated loan, or bank settlement instruction.

21. Frontend / UX Specification

Screen	Must show
Landing	Problem, value proposition, role entry, demo CTA.
Supplier dashboard	Receivables, verification status, financeable state, bids, funding, repayment.
Create receivable	Invoice metadata, buyer, amount, due date, commodity.
Document upload	Document types, hash status, version, upload progress.
Verification report	PASS/WARN/FAIL checks, extracted values, source document reference, engine version.
Buyer review	Invoice summary, document list, verification checks, attestation digest, Accept/Dispute.
Financing marketplace	Financeable receivables, requested amount, offers, selected bid.
Financier detail	Attestation status, buyer acceptance, evidence summary, risk flags, fund action.
Receivable timeline	State changes, timestamps, actors, tx hashes, explorer links.
Tamper test	Original vs modified document, hash comparison, blocked state.

21.1 UX Principles

- Make trust state visible: VERIFIED, ATTESTED, FINANCEABLE, FUNDED, REPAYED.
- Never show “success” before the relevant transaction or backend operation is actually confirmed.
- Every AI flag needs an explanation, not only a score.
- Every blockchain action should show wallet, network, transaction hash and explorer link.
- Use a single canonical “Receivable Timeline” to tell the entire story to a judge.

22. Security Threat Model

Threat	Attack example	Required defense
Document tampering	Change invoice amount after attestation.	Content hash + canonical attestation digest + financeability gate.

Threat	Attack example	Required defense
Duplicate invoice	Submit same invoice ID twice.	Unique invoice constraint + duplicate hash checks + on-chain registry.
Double financing	Two financiers fund the same receivable.	On-chain single-assignment invariant.
Unauthorized acceptance	Non-buyer attempts to accept.	Wallet/role validation in contract and backend.
Private key leakage	Developer commits key to Git.	Environment secrets, ignore files, secret scan, never display keys.
Replay	Reuse old signed acceptance/funding message.	Chain nonce/transaction semantics + explicit state checks.
RPC impersonation / wrong network	App points to wrong chain.	Network identity verification and environment configuration.
AI prompt injection	Malicious PDF contains instructions to AI.	Treat document text as data; structured extraction only; never execute embedded instructions.
Database compromise	Off-chain state is modified.	Blockchain remains source of truth for shared lifecycle; audit logs reveal divergence.

22.1 Smart Contract Security Baseline

- Use OpenZeppelin contracts where appropriate.
- Use access control and role separation.
- Check-effects-interactions pattern for value-moving functions.
- Avoid arbitrary external calls.
- Emit events for all lifecycle state changes.
- Add unit tests for every prohibited transition.
- Run static analysis if time permits.

23. Privacy and Compliance Guardrails

- Store only the minimum shared trust state on-chain.
- Do not upload personal or private business documents to a public blockchain.
- Use test identities and synthetic data for the hackathon.
- No real bank credentials, real financial account information or real customer identity documents are needed.
- Do not describe the prototype as a regulated lending platform.
- Present RWA/receivable representation as a technical prototype for testnet demonstration.

Regulatory interpretation is outside the scope of this build document. Any production deployment would require legal, financial, KYC/AML, data-protection and jurisdiction-specific review.

24. Auditability and Observability

Event	Application log	On-chain proof
Receivable created	Yes	createReceivable tx / event
Document uploaded	Yes	document hash referenced in attestation later
Verification completed	Yes	verification hash / attestation payload
Buyer accepted	Yes	acceptance tx / event
Attestation anchored	Yes	attestation tx / digest
Financing opened	Yes	financing tx / event
Bid selected	Yes	selection state / event
Funding	Yes	fund tx / event
Repayment	Yes	repayment tx / event
Closure	Yes	close event / final state

25. Test Strategy and Acceptance Criteria

25.1 Test Layers

- Unit tests: parsing, normalization, hashing, rule evaluation.
- Backend tests: role checks, idempotency, state transitions.
- Contract tests: allowed and prohibited lifecycle transitions.
- Integration tests: backend -> contract -> MST Testnet.
- UI tests: canonical happy path and tamper path.
- Demo reset test: seed dataset restores the expected state.

25.2 Acceptance Criteria

ID	Acceptance criterion
AC-01	Supplier creates canonical invoice INV-2026-09124 with INR 8,50,000 and 60-day due date.

ID	Acceptance criterion
AC-02	Invoice, PO and GRN can be uploaded and receive deterministic hashes.
AC-03	Verification report shows at least amount, buyer and quantity checks.
AC-04	Buyer acceptance produces a real testnet transaction hash.
AC-05	Attestation digest is anchored on MST and visible via explorer.
AC-06	Only the attested and buyer-accepted receivable appears as financeable.
AC-07	A financier can submit a bid and supplier can select one bid.
AC-08	Funding state appears as FUNDED and is auditable.
AC-09	Repayment state appears as REPAYED and then CLOSED.
AC-10	Changing INR 8,50,000 to INR 9,50,000 causes an integrity mismatch and blocks financeability.
AC-11	No private key or test secret appears in browser source, Git or logs.
AC-12	A fresh machine can reproduce the demo from the repository and documented environment variables.
AC-13	Public GitHub repository contains full source plus README with MST integration details, contract address(es) and setup instructions.
AC-14	Final submission includes at least one deployed MST Testnet contract address and at least one verifiable MST Testnet transaction hash.
AC-15	The demo exposes multiple meaningful MST state-changing transactions or otherwise demonstrates that MST is integral to the product, not merely a wallet connection.
AC-16	BridgeKey integration is tested as the primary wallet path; any fallback EVM wallet is clearly documented as fallback.
AC-17	No fabricated deployment addresses, transaction hashes, explorer links or misleading testnet claims appear anywhere in code, UI or submission artifacts.

26. Judge Demo Script

26.1 Unique Selling Points (USPs)

These are the core differentiators InnoVision should emphasize in the pitch. They are specification-grounded product claims, not promises of production-scale lending or regulatory approval.

USP	Why it matters
MST is a shared trust layer, not a database	MST records shared proof and lifecycle state that supplier, buyer, financier and platform must independently verify; sensitive documents remain off-chain.
AI is advisory; humans attest	AI extracts, compares and flags. A business participant provides the binding acceptance signature; AI never releases funds by itself.
Live tampering detection	Changing INR 8,50,000 to INR 9,50,000 changes the exact-file hash and canonical digest, causing INTEGRITY MISMATCH and blocking financeability.
Supabase-first speed without sacrificing architecture	Supabase consolidates Auth, Postgres, Storage, Realtime and Edge Functions, freeing the team to focus on verification, blockchain and the demo.
Closed trust-to-finance lifecycle	Created -> Verified -> Attested -> Buyer Accepted -> Financeable -> Funded -> Outstanding -> Repaid -> Closed.
On-chain proof, off-chain privacy	Only hashes, identifiers, wallets, state and events needed for shared trust go to MST; raw evidence remains in protected Supabase Storage.
Anti-double-financing at protocol level	Once a receivable is funded, contract invariants prevent a second financier from funding the same receivable.
Traceability beyond the invoice	Farm -> Batch -> Quality Certificate -> Exporter -> Invoice -> Buyer -> Financier -> Repayment.

Target demo length: 120-180 seconds. The goal is not to show every feature. The goal is to show one business story, multiple real MST state-changing transactions, and one attack.

1. Opening (10 sec): “A Basmati exporter is owed INR 8.5 lakh in 60 days but needs working capital today.”
2. Supplier (20 sec): Create INV-2026-09124 and upload Invoice + PO + GRN.
3. AI verification (20 sec): Show extracted values and PASS checks; risk example 18/100 LOW.
4. Buyer (15 sec): Buyer reviews and signs acceptance; show confirmed MST transaction.
5. Attestation (10 sec): Show ATTESTED / FINANCEABLE state, the attestation digest and its confirmed MST transaction.

6. Financing (20 sec): Show 2-3 fictional testnet financier offers; select one and fund.
7. Repayment (15 sec): Buyer repays; show REPAID / CLOSED.
8. Attack (25 sec): Modify invoice from INR 8,50,000 to INR 9,50,000; upload; show hash mismatch and financing blocked.
9. Close (10 sec): “We did not put documents on-chain. We put the trust state and financial lifecycle on-chain.”

DEMO RULE

Never demo with a live dependency that can fail without a fallback. Keep a fully seedable local database and pre-verified testnet contract deployment, but still execute at least one live MST transaction during the presentation when possible.

27. 24-Hour Delivery Plan

Time	Workstream	Deliverable	Owner
0-1.5 h	MST + Supabase setup	Verify MST testnet identity, BridgeKey, faucet and MSTScan; install @mstblockchain/mst-sdk; test VibeKit bootstrap; create public GitHub + Supabase project; initial migrations; deploy skeleton contract.	Blockchain + Backend
1.5-4 h	Contract core	InvoiceRegistry + AttestationRegistry; lifecycle invariants; unit tests.	Blockchain
2-6 h	Supabase foundation	profiles, organizations, invoices, documents, attestations; RLS; Storage buckets; seed/reset script.	Backend
4-8 h	AI verification	Python FastAPI service; PyMuPDF/OCR extraction; typed schema; deterministic checks; verification report.	AI/Backend
5-10 h	Frontend	React + Vite dashboards for Supplier, Buyer and Financier; Supabase Auth; timeline/status UI.	Frontend
8-12 h	Integration	React -> Supabase -> AI service -> wallet -> MST contract; receipt handling; Realtime status.	Integration
12-14 h	Financing + repayment	Bid flow, selection, FundingPool, repayment and closure.	Blockchain + Backend

14-16 h	Tamper attack	Document versioning, hash mismatch, blocked financeability, UI attack path.	Integration
16-18 h	Polish	Visual hierarchy, explorer links, loading/error states, traceability view if time permits.	Frontend/Product
18-20 h	Security + test	RLS tests, contract state tests, secret scan, duplicate-financing attempt, reset script.	All
20-22 h	Dress rehearsal	Run exact judge script twice; verify public GitHub, contract address, real transaction hash(es), MSTScan links, demo link; capture backup video and screenshots.	All
22-24 h	Freeze	No risky new features; final bug fixes; submission artifact verification; re-check no fabricated chain data and README completeness.	All

28. Team Responsibilities

Role	Ownership	Definition of done
Developer 1 - Lead / Full-Stack + Blockchain (60%)	MST, Solidity/Hardhat, wallet/MSTScan, React/Vite frontend, core UX, architecture, integration, deployment and demo.	Core contracts deploy; wallet and MSTScan work; frontend connects to Supabase/API and chain; happy path and demo run end-to-end.
Developer 2 - Backend + AI / Supabase (40%)	Supabase DB/Auth/Storage/RLS/Realtime/Edge Functions, FastAPI AI, OCR, verification rules, hashing/tamper detection and APIs.	Schema/RLS pass tests; upload + extraction + PASS/WARN/FAIL checks work; tampering is blocked; APIs are integrated and stable.

29. Deployment and Runbook

29.1 Environment Variables

```
[env]
# MST Blockchain
MST_RPC_URL=<OFFICIAL_MST_TESTNET_RPC>
MST_CHAIN_ID=<CONFIRMED_BY_MST_OR_ORGANIZERS>
MST_EXPLORER_URL=https://mstscan.com
```

```

DEPLOYER_PRIVATE_KEY=<LOCAL_SECRET_ONLY>

# Supabase - frontend
VITE_SUPABASE_URL=<SUPABASE_PROJECT_URL>
VITE_SUPABASE_ANON_KEY=<SUPABASE_ANON_KEY>

# Supabase - Edge Function/server secrets only
SUPABASE_SERVICE_ROLE_KEY=<SERVER_SIDE_SECRET_ONLY>
AI_SERVICE_URL=<PYTHON_AI_SERVICE_URL>
AI_SERVICE_KEY=<SERVER_SIDE_SECRET_ONLY>

# Contracts
INVOICE_REGISTRY_ADDRESS=<DEPLOYED_ADDRESS>
ATTESTATION_REGISTRY_ADDRESS=<DEPLOYED_ADDRESS>
FINANCING_POOL_ADDRESS=<DEPLOYED_ADDRESS>
REPAYMENT_MANAGER_ADDRESS=<DEPLOYED_ADDRESS>
MST_NETWORK=testnet

```

Do not replace the placeholders above with guessed values. The organizer guide now supplies the official faucet, documentation, SDK, VibeKit, MCP and MSTScan resources. The current MST Python SDK documentation resolves the testnet RPC to <https://testnetrpc.mstblockchain.com>; treat that as the current SDK-documented default, verify network identity and connectivity at build time, and keep `MST_CHAIN_ID` explicit until confirmed. Supabase project URL/keys come from project settings; the service-role key MUST remain server-side. BridgeKey is the primary wallet target because the organizer strongly recommends it; use an approved EVM wallet only as fallback.

29.2 Deployment Sequence

1. Create/confirm the public GitHub repository and README skeleton; install the official MST TypeScript SDK and test VibeKit bootstrap tooling.
2. Install/test BridgeKey as the primary wallet path; create or import a dedicated testnet wallet.
3. Obtain test MSTC from <https://faucet.masterstroke.academy>.
4. Verify MST Testnet network identity, RPC connectivity, chain ID and <https://mstscan.com> before deployment.
5. Deploy the MVP contract(s) in a deterministic order and record the real deployed address(es).
6. Execute at least one real state-changing transaction immediately after deployment; capture the real transaction hash and explorer link.
7. Save verified addresses/hashes to a versioned deployment manifest and environment configuration; never fabricate them.
8. Seed the canonical AgriCredX dataset and run end-to-end acceptance tests.
9. Finalize README with MST integration details, contract address(es), transaction hash(es), setup instructions, verified official-resource links actually used, and demo link before submission.

30. Roadmap and Future Extensions

Phase	Extension	Purpose
Phase 1 - Buildathon	Canonical MVP on MST Testnet	Prove the end-to-end trust-to-finance flow.
Phase 2	Digital Product Passport / traceability graph	Connect farm, batch, quality and logistics evidence.
Phase 3	ERP / logistics connectors	Reduce manual document upload.
Phase 4	Automated evidence attestations	Allow trusted systems to attest delivery and acceptance events.
Phase 5	More financing sources	Expand marketplace liquidity and configurable risk policies.
Phase 6	ZK / selective disclosure	Prove selected claims without exposing sensitive business details, subject to mature implementation.

31. Reference Architecture and External Sources

This section distinguishes the user-provided AgriCredX concept from current external verification of MST/BridgeKey ecosystem capabilities.

Implementation stack decision for InnoVision: Supabase is the preferred and primary application backend, following the source material that explicitly highlights Supabase for fast, low-friction Auth, Postgres, Storage and Realtime setup during the 24-hour build.

31.1 User-Provided Source Basis

Primary source: uploaded AgriCredX specification PDF. The source defines the project as blockchain-backed attested receivables financing for agricultural supply chains, the canonical INR 8.5 lakh Basmati rice scenario, the multi-party workflow, the four logical contracts, the architecture, tech stack, demo script and 24-hour plan. The source explicitly states that sensitive documents remain off-chain while MST records proof and lifecycle state.

31.2 Current MST / Wallet Verification

Official MST Buildathon Track Guide + MST official resources: <https://docs.mstblockchain.com> - treat the organizer-provided documentation as authoritative for MST development and network-specific values.

MST Testnet Faucet: <https://faucet.masterstroke.academy> - official organizer-provided faucet for test MSTC.

MSTScan Explorer: <https://mstscan.com> - organizer-provided explorer for verifying live MST transactions and contract activity. Do not invent a testnet-specific explorer path; resolve the appropriate network view at build time.

MST TypeScript SDK: <https://www.npmjs.com/package/@mstblockchain/mst-sdk> | MST Python SDK: <https://pypi.org/project/mst-sdk-python/> | VibeKit: <https://www.npmjs.com/package/@mstblockchain/mst-vibe-kit>

The official SDKs and VibeKit are permitted developer resources; use the TypeScript SDK as the preferred MST-specific client layer, keep the Python SDK optional and outside the AI runtime, and use VibeKit as a bootstrap/deployment accelerator.

BridgeKey: <https://bridgekey.io/> | Chrome Extension: <https://chromewebstore.google.com/detail/bridgekey/bfjojdcfenehemjgglepdjomkpginlkg>

The organizer guide strongly recommends BridgeKey. The current BridgeKey site describes it as non-custodial, MST-native and EVM multi-chain. Use it as the primary wallet target.

BridgeKey Android app is also listed by the organizer; it is out of scope for the web MVP but may be useful for wallet fallback/testing.

MST MCP endpoint: <https://mcp.mstblockchain.com/sse> | Official docs: <https://docs.mstblockchain.com>

Use the MCP endpoint only as a developer-time documentation aid. Do not place organizer-provided MCP credentials in source control.

31.3 Source-Derived Claims vs. Implementation Recommendations

Category	Included content
Product/source-derived	Business problem, canonical case, logical contract responsibilities, attestation concept, financing lifecycle, AI-as-assistant principle, on-chain/off-chain split and Supabase-first architecture.
Current external verification	Current organizer resources confirm official docs, faucet, MSTScan, BridgeKey, TypeScript SDK, Python SDK, VibeKit and MCP links. The current Python SDK documents testnet RPC https://testnetrpc.mstblockchain.com .
Implementation recommendation	Use strict state invariants, typed AI output, deterministic hashing, role checks, testnet-only demo, seedable dataset, official MST SDK/BridgeKey, real on-chain proof, and a 120-180 second judge flow.
Organizer-guide-derived	Mandatory meaningful MST usage, MST Testnet deployment before submission, public GitHub repository, contract address + verifiable transaction hash, working demo expectation, BridgeKey recommendation, and official SDK/VibeKit/MCP resources.

32. Final Antigravity Build Checklist

BUILD IN THIS ORDER

1) Chain + contracts -> 2) DB + APIs -> 3) AI verification -> 4) supplier/buyer flow -> 5) financing -> 6) repayment -> 7) tamper attack -> 8) polish. Do not start by building every screen.

- ☐ Is MST usage meaningful and integral (not just wallet connection / isolated transaction)?
- ☐ Is the project deployed on MST Testnet before submission?
- ☐ Do we have at least one real, verifiable MST Testnet transaction hash?
- ☐ Do we have the real deployed MST Testnet contract address?
- ☐ Is the public GitHub repository complete with MST integration README?
- ☐ Has BridgeKey been tested as the primary wallet path?
- ☐ Have the official MST TypeScript SDK and VibeKit been evaluated/used where they reduce risk?
- ☐ Do we have a verified MST Testnet RPC and chain identity?
- ☐ Do we have test MSTC from the official faucet?
- ☐ Is a contract deployed and visible in the explorer?
- ☐ Can one canonical receivable be created from seeded data?
- ☐ Do invoice/PO/GRN hashes reproduce deterministically?
- ☐ Does verification show explicit PASS/WARN/FAIL reasons?
- ☐ Can the buyer sign acceptance?
- ☐ Can the attestation digest be recomputed from stored inputs?
- ☐ Can the financing state be blocked for an unattested/unaccepted receivable?
- ☐ Can only one financier fund the receivable?
- ☐ Can repayment move the lifecycle to REPAID and CLOSED?
- ☐ Does changing INR 8,50,000 to INR 9,50,000 trigger integrity mismatch?
- ☐ Can every blockchain action be opened in MSTScan?
- ☐ Are all secrets outside source control?
- ☐ Can the demo be restored from a seed/reset script?
- ☐ Has the exact 120-180 second judge script been rehearsed twice?

Appendix A - Canonical Demo Data

[json]

```
{
  "invoiceId": "INV-2026-09124",
  "commodity": "Basmati Rice",
  "supplier": "Demo Basmati Exporter",
  "buyer": "ABC Foods",
  "invoiceAmount": 850000,
  "currency": "INR",
  "paymentTermDays": 60,
  "requestedFinancing": 820000,
  "qualityCertificate": true,
  "poMatched": true,
  "grnMatched": true,
  "buyerAccepted": true,
  "duplicateCheck": false,
```

```

"expectedRiskScore": 18,
"expectedRiskBand": "LOW"
}

```

Appendix B - Example Repository Structure

```

[text]
agicredx/
  apps/
    web/          # React + Vite + TypeScript frontend
  supabase/
  functions/
    verify-receivable/  # invokes Python AI service
    audit-chain-event/  # privileged status/audit coordination
  migrations/
  seed.sql
  config.toml
  services/
    ai/           # Python FastAPI AI microservice
    app/
    tests/
  contracts/
  src/
    InvoiceRegistry.sol
    AttestationRegistry.sol
    FinancingPool.sol
    RepaymentManager.sol
  test/
  scripts/
  packages/
    shared-types/
    chain-client/
    hashing/
  data/
  demo/
    invoice.pdf
    po.pdf
    grn.pdf
    quality.pdf
  docs/
    PRD_SRS.md
    CONTRACTS.md
    DEMO_RUNBOOK.md
  .env.example
  README.md

```

Appendix C - Final Product Statement

AGRICREDX

AgriCredX transforms a fragmented agricultural trade record into a verifiable, buyer-attested and financeable receivable. AI helps verify the evidence; MST provides the shared trust and lifecycle state; the financing marketplace turns verified trust into working capital.

Buildathon compliance note: final submission artifacts MUST contain the public GitHub repository, real MST Testnet contract address(es), at least one verifiable transaction hash, working demo/application link where applicable, and demo video/backup recording. Use only values produced by the deployed system; never substitute fabricated addresses, hashes or explorer links.

End of Specification